

PhotoMedicine Labs

OCTA Vessel Restoration & Segmentation

A two-stage deep learning pipeline for retinal vessel enhancement and binary segmentation

ResU-Net

OCTA-500 + ROSE

TensorFlow 2.17

Docker



The Problem with Clinical OCTA

Motion artifacts degrade image quality

OCTA acquisitions require patients to maintain fixation for 2–6 seconds. Microsaccades and involuntary eye movements create banding artifacts that obscure fine vessel detail — especially capillaries.

Manual quality control is the bottleneck

Clinicians must manually reject or re-acquire degraded scans. In high-throughput settings this is impractical and introduces inter-rater variability.

Segmentation remains challenging

Vessel-to-background ratio is ~10–15%. Class imbalance causes naive models to predict all-background and still achieve low loss.

~15%

vessel pixels
per image

2–6 s

fixation time
required

7:1

background-to-
vessel ratio

Two-Stage Pipeline: Option C

Restore first, then segment — decoupled objectives yield cleaner signals for each model



Eventually: fine-tune on chicken embryo OCTA data (Andrei's lab)

Why Option C (restore → segment) rather than joint training?

- Decoupled training: restoration model optimises PSNR/SSIM without being confused by segmentation loss
- At inference, degraded real-world data passes through Stage 1 first — the segmentation model always sees clean-looking inputs
- Each stage can be independently benchmarked, replaced, or fine-tuned without retraining the other

Dataset: OCTA-500 + ROSE Combined

PRIMARY

OCTA-500

500 retinal OCTA volumes

400 × 400 pixel resolution

ILM_OPL slab projection used

GT_LargeVessel ground truth labels

500 subjects, one eye each
IEEE DataPort open access

80 / 10 / 10 train / valid / test

SUPPLEMENTARY

ROSE

~151 additional images

2 scanners Optovue + Heidelberg

304→400 upsampled resolution

Capillary-level vessel annotations

ROSE-1 (SVC_DVC) + ROSE-2
Zenodo open access

80 / 20 train / valid (no held-out test)

OCTA-500 alone showed val_loss instability — ROSE adds scanner diversity, finer vessel annotations, and better generalisation

Data Pipeline: data.py

01

Load & Resize

PIL LANCZOS resize to 400×400. Masks use NEAREST to preserve binary edges. Supports .png, .tif, .tiff, .bmp, .jpg.

02

CLAHE Enhancement

Contrast Limited Adaptive Histogram Equalisation (clipLimit=2.0, tileGridSize=8×8). Enhances local vessel contrast without amplifying background noise. Done once at preprocessing — not per epoch. Validated in Giarratano et al. 2020.

03

Binarise Masks

GT_LargeVessel labels thresholded to {0,1} — required for BinaryCrossentropy and Dice loss. Saves as uint8 to skip the /255 step in the generator.

04

Save as .npy

NumPy .npy format: 3-5× faster than PIL Image.open() in training loops. Exact array values preserved (no JPEG compression artifacts). Metadata JSON saved alongside: vessel_ratio, shape, sample_id.

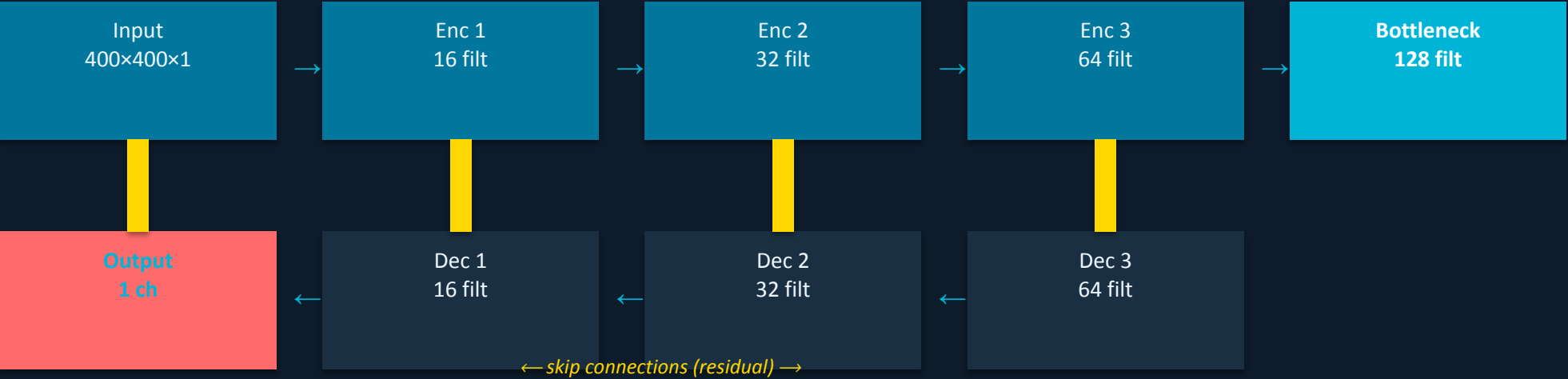
05

Train / Valid / Test Split

OCTA-500: 80/10/10 stratified split (random_state=28). ROSE: 80/20 train/valid — no test contribution (OCTA-500 is the held-out benchmark).

ResU-Net Architecture

Shared backbone for both Stage 1 (restoration) and Stage 2 (segmentation) — output head differs



Hyperparameters

Filters	(16, 32, 64, 128)	LR decay	0.95 × per 10,000 steps
Kernel	3×3	Batch size	4 (GPU memory constraint)
Dropout	0.3 @ bottleneck + decoder	Epochs	400 max + EarlyStopping patience=20
Optimizer	Adam lr=1e-4 $\beta_1=0.8$ $\beta_2=0.999$		

Loss Functions & Training Strategy

STAGE 1 — RESTORATION

$$L_{\text{total}} = L_2 + \lambda \cdot L_{\text{perceptual}}$$

L_2 (MSE): pixel-wise fidelity loss — penalises large pixel errors

$L_{\text{perceptual}}$: VGG feature-space loss — preserves vessel texture and edges

Metrics: PSNR > 23 dB, SSIM > 0.55 (Liao et al. 2022 baseline)

STAGE 2 — SEGMENTATION

$$L_{\text{total}} = \text{Dice} + 0.3 \times \text{BCE} + \text{clDice}$$

Dice: handles class imbalance (vessels ~10–15%) — dominant term

BCE (weight=0.3): pixel-level probability calibration

clDice: centerline-aware loss — penalises broken vessel topology

BCE_WEIGHT=0.5 was too high → blurry masks → reduced to 0.3

Data Augmentation Strategy

Motion Artifact Simulation (Stage 1)

- Severity: 0.30–0.70 (few-repeat → 2-repeat acq.)
- B-scan dropout rate: 0.4 (horizontal banding)
- Speckle noise: Rayleigh-distributed, scale=0.2
- Shot noise: Poisson photon fluctuations, scale=0.3
- Additional Gaussian $\sigma=0.4$ on top

Geometric Augmentation (Stage 2)

- Flip LR / UD: 50% each
- Rotation: $\pm 10^\circ$
- Scale: 0.95–1.05×
- Translate: $\pm 10\%$
- Elastic deformation: disabled (vessels are rigid)

Evaluation Metrics

Dice (F1)

$$2 \cdot TP / (2 \cdot TP + FP + FN)$$

Primary metric for medical image segmentation. Harmonic mean of precision and recall.

Precision

$$TP / (TP + FP)$$

Of predicted vessel pixels, how many are correct? Low → over-segmentation.

IoU / Jaccard

$$TP / (TP + FP + FN)$$

Stricter than Dice — larger denominator. Same ordering, lower absolute values.

Recall

$$TP / (TP + FN)$$

Of true vessel pixels, how many found? Low → under-segmentation.

clDice

Centerline-aware Dice

Penalises broken vessel centerlines. Critical for OCTA: a disconnected map is clinically misleading even if pixel-Dice looks good.

PSNR / SSIM

$$dB / [0,1]$$

Restoration quality. PSNR: pixel fidelity. SSIM: structure + texture. Target: PSNR > 23 dB, SSIM > 0.55.

Repository Structure

Data			config.py	Central hyperparameters — filters, LR, augmentation, loss weights	test.py	Single-image inference demo — load model, predict, save mask	Dockerfile	NVIDIA NGC TF 2.17 base, pip install, workspace mount
	data.py	OCTA-500 + ROSE loading, CLAHE, train/valid/test split, .npy save	Eval		test2.py	Validation set quick-check: prediction stats vs ground truth	docker-compose.yml	GPU reservation, ipc=host, volume mount, ulimits
Model			evaluate_1.py	Full test-set evaluation: metrics table, segmentation visuals, overlays	Utils		entrypoint.sh	python model.py && python evaluate.py
	model.py	ResU-Net build, loss functions (Dice, BCE, cIDice), training loop	evaluate_2.py	Restoration model evaluation: PSNR/SSIM on test set		keras_to_h5.py	Convert Keras 3 .keras weights → legacy .h5 for compatibility	
	model-0_9.py	Previous model version — earlier architecture iteration for reference	QA			read_tf_events.py	Read TensorBoard logs, plot training curves, mark best epoch	
Config			check.py	Data integrity checks: shapes, dtypes, vessel ratios, visualisation	Infra			

Key folders:

benchmarks/

docs/

images/

data/

utils/

Docker + GPU Setup

Dockerfile

Base image: nvr.io/nvidia/tensorflow:24.01-tf2-py3
→ TF 2.17 pre-built with CUDA drivers, cuDNN, TensorRT

pip install -r requirements.txt inside container
→ keras≥3.0, matplotlib, scikit-learn, imgaug, tqdm...

WORKDIR /workspace/photomedicine_labs
COPY requirements.txt + entrypoint.sh
ENTRYPOINT: python model.py && python evaluate.py

docker-compose.yml

Volume: . → /workspace/photomedicine_labs
→ Live code sync — edit .py files without rebuild

ipc: host
→ Shared memory for multi-GPU DataLoader

GPU: driver=nvidia, count=all

ulimits: memlock=-1 (disable memory locking limit)
ulimits: stack=64MB

Quick start (from project root):

docker compose up --build

PhotoMedicine Labs · URA

OCTA ROI Selector & Galvo Waveform Generator

PyQt5 desktop application — interactive ROI selection on OCTA images,
real-time galvo voltage computation, vessel segmentation overlay, waveform export

PyQt5

TensorFlow

Matplotlib

ResU-Net

NumPy / OpenCV



What Does This Tool Do?

The Clinical Need

In targeted OCTA re-acquisition, the galvo mirrors must be driven to scan only a sub-region of interest rather than the full retina. This requires converting pixel coordinates on the reference image directly into galvo voltage waveforms.

Previously: manually computing voltage ranges from pixel coordinates — error-prone and slow. The ROI GUI automates this entirely.

OCTA ROI SELECTOR — CORE FEATURES

Interactive ROI Drawing

Drag to select any rectangular sub-region on a loaded OCTA image. Multiple ROIs accumulate. Corner handles, voltage labels, semi-transparent fill.

Live Galvo Voltage Readout

Instant px→V conversion on mouse release. X (fast/sawtooth) and Y (slow/stair-step) voltages displayed in real-time alongside pixel coordinates.

Waveform Preview + Export

Matplotlib live preview of sawtooth + stair-step waveforms. Export full waveforms as .npy or .csv for your DAQ system.

Vessel Segmentation Overlay

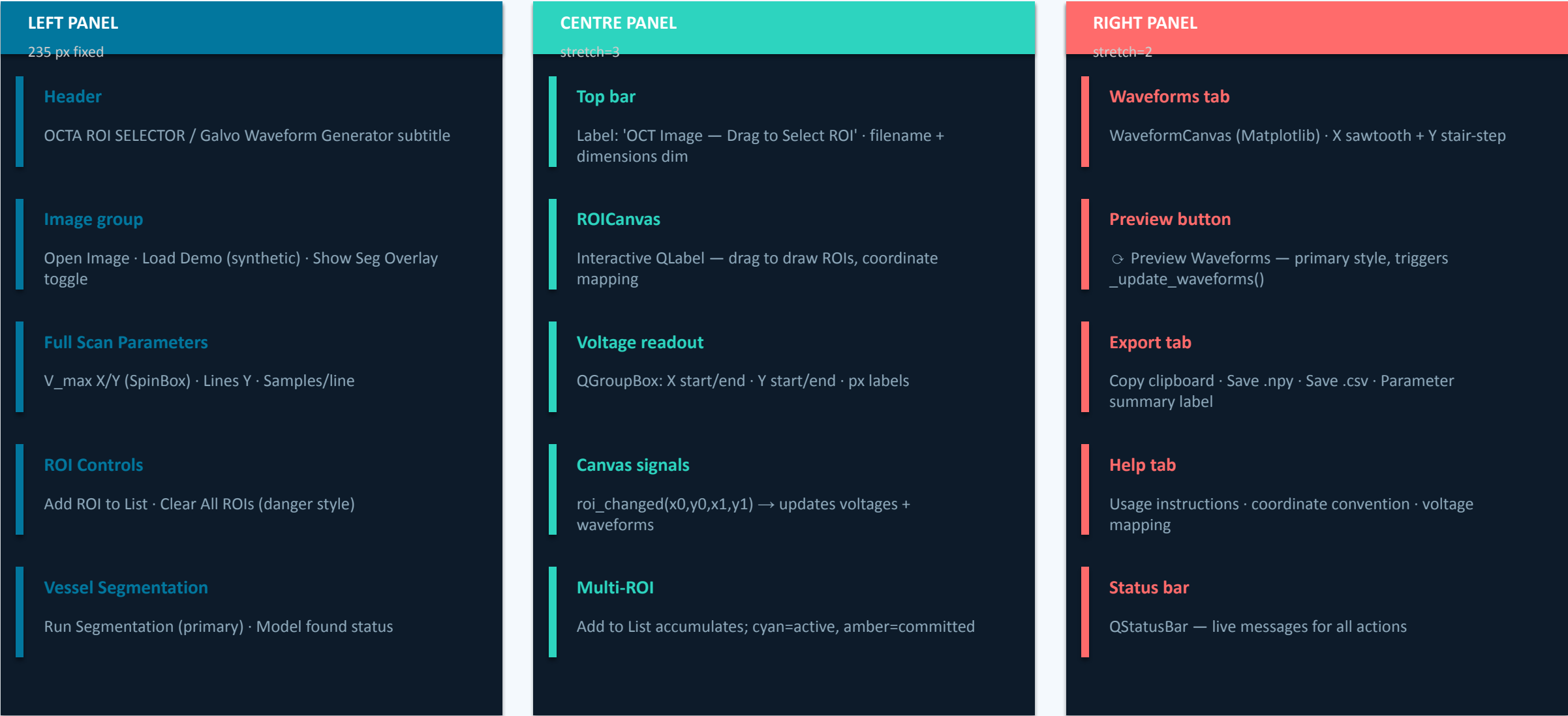
Runs trained ResU-Net (.keras) directly — no Docker. Background QThread keeps UI responsive. Cyan overlay composited on image at 70% opacity.

Dark Scientific UI

Monospace Courier New theme. All parameters editable: V_max, scan lines, samples/line. Status bar, tab panel (Waveforms / Export / Help).

Three-Panel Window Architecture

OCTROISelector (QMainWindow) · min 1200×760 px · dark scientific theme throughout



ROICanvas — Interactive Image Widget

roi_canvas.py · subclasses QLabel · emits roi_changed(x0, y0, x1, y1) and roi_cleared()

Coordinate Mapping

Three coordinate spaces:

● Label space

Raw mouse event pixels (QPoint from Qt)

● Scaled space

Pixmap after KeepAspectRatio scaling — for drawing

● Image space

Original image pixels — what gets emitted in signals

Key mapping functions:

```
# Label px → image px
def _label_to_img(pt, scaled_size):
    off = _scaled_offset(scaled_size) # letterbox offset
    sx = orig_w / scaled_w
    sy = orig_h / scaled_h
    return QPoint(int((pt-off).x * sx), int((pt-off).y * sy))

# Image QRect → scaled space (for drawing overlays)
def _img_to_scaled(img_rect, scaled_size):
    off = _scaled_offset(scaled_size)
    sx = scaled_w / orig_w
    return QRect(off.x + img_rect.x*sx, ...)
```

Drawing Pipeline

<code>mousePressEvent</code>	Set <code>_drawing=True</code> , record <code>_roi_start</code>
<code>mouseMoveEvent</code>	Update <code>_roi_end</code> → <code>_update_display()</code> live
<code>mouseReleaseEvent</code>	Map label→image coords, clamp to bounds, emit <code>roi_changed()</code>
<code>_update_display()</code>	Scale pixmap → composite seg overlay → draw all ROIs
<code>_draw_roi()</code>	QPen rect + corner handles + voltage label text

Signals & Public API

<code>roi_changed(x0,y0,x1,y1)</code>	Emitted on mouse release — image px coords
<code>roi_cleared()</code>	Emitted when Clear All ROIs clicked
<code>load_image(path)</code>	Load from file path → QPixmap
<code>load_ndarray(arr)</code>	Accept HxW or HxWx3 uint8 numpy array
<code>set_segmentation_overlay(arr_rgba)</code>	HxWx4 RGBA numpy → semi-transparent composite
<code>toggle_segmentation(bool)</code>	Show/hide overlay without rerunning inference
<code>add_roi_to_list()</code>	Commit active ROI to accumulated list (amber)
<code>clear_rois()</code>	Clear all ROIs + emit <code>roi_cleared()</code>

Pixel → Voltage → Waveforms

waveforms.py · pure functions, no Qt imports — fully testable in isolation

px_to_volts() — Linear Proportional Mapping

$$v = v_{\min} + (px / \text{img_size}) \times (v_{\max} - v_{\min})$$

px = 0 → 0.000 V (image left edge)
px = 200 → 1.000 V (image centre)
px = 400 → 2.000 V (image right edge)

roi_to_voltages() — Full ROI Conversion

```
roi_to_voltages(x0=217, y0=246, x1=352, y1=337, img_w=400, img_h=400)
→ { vx0: 1.085, vx1: 1.760, vy0: 1.230, vy1: 1.685 }
```

Calls `px_to_volts()` for each of the 4 ROI corners independently

`volts_to_px()` is the inverse — converts voltage back to pixel for display

X AXIS — SAWTOOTH

Fast axis · sweeps left→right each line

```
def generate_sawtooth(
    v_start, v_end,
    n_lines, samples_per_line):

    line = np.linspace(
        v_start, v_end,
        samples_per_line)
    return np.tile(line, n_lines)

# shape: (n_lines × spl,)
# fly-back: DAQ handles retrace
```

Preview: first 50 lines only
for rendering speed

Export: full `n_lines` waveform
(up to 2000 lines, 5000 spl)

Total samples = `n_lines × samples_per_line`

Y AXIS — STAIR-STEP

Slow axis · one step per scan line

```
def generate_stair_step(
    v_start, v_end,
    n_lines, samples_per_line):

    steps = np.linspace(
        v_start, v_end,
        n_lines)
    return np.repeat(
        steps, samples_per_line)

# shape: (n_lines × spl,)
# constant voltage per line
```

Holds voltage constant for
`samples_per_line` samples

Steps linearly from `vy0`→`vy1`
across entire ROI height

`np.repeat()` — not tile — one level per line

In-App Vessel Segmentation

segmentation.py · no Docker · no training environment · .keras is self-contained

No Docker required. The segmentation model
(segmentation_best.keras)
is loaded directly from the models/ directory for CPU
inference.

Inference Pipeline

1 Lazy Load

Model loads on first predict() call — GUI starts fast even if TF takes 3s

2 Preprocess

Any input format → grayscale → cv2.resize 400×400 → float32 /255 → (1,400,400,1)

3 Inference

model.predict(inp, verbose=0) → prob_map shape (400,400), values [0,1]

4 Postprocess

Threshold 0.5 → binary mask at model resolution

5 Resize Back

cv2.INTER_NEAREST resize → original image dimensions (preserves binary values)

6 RGBA Overlay

Vessel pixels → [57, 208, 216, 180] (cyan #39D0D8 at 70% opacity)

Key Details

No Docker required

segmentation_best.keras is fully self-contained — weights + architecture in one file

CPU inference ~1s

GPU used automatically if available via TF device detection

is_available() check

GUI checks model path on startup and shows ✓/⚠ status label

predict_with_metrics()

Optional Dice + IoU computation against a ground truth mask for evaluation workflows

Preprocessing match

Exact same pipeline as training: resize→normalise→batch+channel dims. Mismatch here would silently degrade predictions.

RGBA overlay design

Background fully transparent (alpha=0). Vessels cyan at alpha=180 ≈70%. Composited in ROICanvas at opacity=0.55 via QPainter.

SegWorker (QThread) — Non-blocking Inference

```
class SegWorker(QThread):
    done = pyqtSignal(object, object) # mask, overlay — emitted on completion
    error = pyqtSignal(str) # emitted on exception

    def run(self):
        mask, overlay = self._model.predict(self._img) # ~1s CPU
        self.done.emit(mask, overlay) # back on main thread via signal

# Button disabled + '⌚ Running...' text while worker is alive
# UI stays fully responsive — user can still pan/zoom during inference
```


WaveformCanvas & Theme System

waveform_canvas.py · theme.py · constants.py

WaveformCanvas (FigureCanvasQTagg)

- **Two-subplot Matplotlib figure** Top: X sawtooth (cyan) · Bottom: Y stair-step (amber)
- **Dark theme matched** facecolor=BG_DARK, axes facecolor=BG_PANEL, spines=BORDER, ticks=TEXT_DIM
- **Voltage reference lines** axhline at v_start (green dashed) + v_end (red dashed) with legend
- **Preview cap** max 50 lines rendered for speed — full export uses actual spin_nlines value
- **Graceful fallback** HAS_MPL flag — if matplotlib absent, renders plain QLabel with install instructions
- **update_waveforms()** Calls generate_sawtooth() + generate_stair_step() → replots both axes → self.draw()

Export Tab — 3 Formats

Copy Clipboard

Copies lbl_summary text — includes pixel coords, voltage ranges, sawtooth/stair-step V0→V1

Save .npy

np.save({'x_sawtooth': xw, 'y_stairstep': yw}) — preserves float64 exact values for DAQ

Save .csv

np.savetxt with header x_sawtooth_V,y_stairstep_V — column stack of both waveforms

theme.py — Qt Stylesheet

- QMainWindow, QWidget → BG_DARK + Courier New monospace
- QGroupBox → border, rounded, ACCENT_CYAN title
- QPushButton → 3 variants: default / #primary (cyan fill) / #danger (red border)
- QLabel#voltage → ACCENT_GREEN bold 13pt for V readouts
- QLabel#header → ACCENT_CYAN 15pt bold letter-spacing
- QSlider, QCheckBox, QTabBar → all custom styled
- Applied once: app.setStyleSheet(STYLESHEET)

constants.py — Single Source of Truth

```
GALVO_V_MIN      = 0.0      # start of full sweep
GALVO_V_MAX      = 2.0      # end of full sweep
OCTA_PIXEL_SIZE  = 500      # native OCTA-500 resolution
MODEL_INPUT_SIZE = 400      # must match training exactly

SEG_MODEL_PATH = MODELS_DIR / 'segmentation_best.h5'

BG_DARK   = '#0d1117'  ACCENT_CYAN = '#39d0d8'
BG_PANEL  = '#161b22'  ACCENT_GREEN = '#3fb950'
BG_CARD   = '#1c2128'  ACCENT_AMBER = '#e3b341'
```

Repository Structure

Clean separation of concerns — UI wiring, math, inference, display all in separate modules

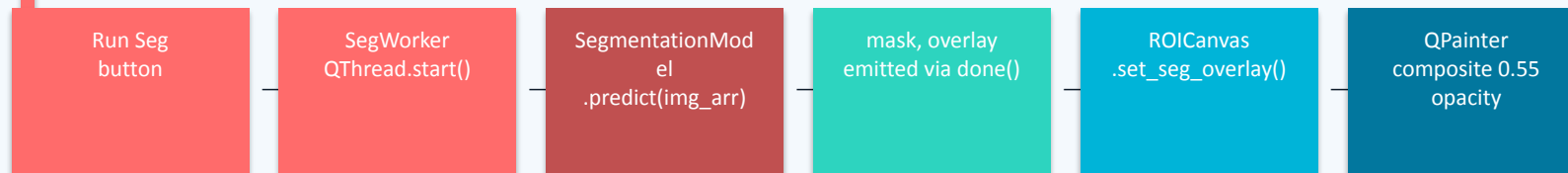
<code>main.py</code> Entrypoint argparse (--image flag) · QApplication setup · HiDPI scaling attributes · window.show() + optional startup image load	<code>main_window.py</code> OCTROISelector All Qt wiring — <code>_build_left/centre/right()</code> · SegWorker thread management · <code>_on_roi_changed()</code> slot · export methods · status bar updates
<code>roi_canvas.py</code> ROICanvas Interactive QLabel · drag-to-draw ROIs · 3-space coordinate mapping · segmentation overlay compositing · multi-ROI accumulation	<code>waveform_canvas.py</code> WaveformCanvas Matplotlib FigureCanvasQTAgg · two-panel sawtooth/stair-step preview · dark-themed axes · graceful HAS_MPL fallback
<code>waveforms.py</code> Pure Math <code>px_to_volts()</code> · <code>volts_to_px()</code> · <code>generate_sawtooth()</code> · <code>generate_stair_step()</code> · <code>roi_to_voltages()</code> · no Qt imports — fully unit-testable	<code>segmentation.py</code> SegmentationModel Lazy TF model load · grayscale/resize/normalise preprocessing · <code>predict()</code> + <code>predict_with_metrics()</code> · RGBA overlay builder
<code>constants.py</code> Single Source of Truth All shared values: voltages, paths, pixel sizes, theme hex colours. Import from here everywhere — never hardcode elsewhere.	<code>theme.py</code> Qt Stylesheet Complete STYLESHEET string for <code>app.setStyleSheet()</code>. All widget variants: <code>default</code> / <code>#primary</code> / <code>#danger</code> / <code>#voltage</code> / <code>#header</code> / <code>#dim</code>

End-to-End Signal & Data Flow

From image load → ROI draw → voltage compute → waveform generate → export



Segmentation branch (parallel, background thread):



Design Decisions

Centre built first

main_window._build_ui() builds centre before left — left panel references self.canvas which must exist first

Letterbox offset

_scaled_offset() accounts for KeepAspectRatio padding — without this, ROI coords are wrong on non-square images

Lazy TF load

TF import inside _load() — avoids 2-3s startup delay even when segmentation isn't used in a session

QThread for seg

model.predict() blocks for ~1s on CPU — running on main thread would freeze the entire UI, no mouse events

INTER_NEAREST resize

Binary mask must use nearest-neighbour interpolation when resizing back — linear would create non-binary float values

Constants module

All magic numbers in constants.py — voltage range, model path, image size. GUI and math code never hardcode values

Setup, Usage & Next Steps

Installation

```
pip install PyQt5 numpy matplotlib Pillow
      opencv-python tensorflow

# Place model in:
# photomedicine-labs/models/segmentation_best.h5

python main.py
python main.py --image data/octa_sample.bmp
```

How To Use

- 1 Load Image** Open Image (PNG/BMP/TIFF) or Load Demo (synthetic vessel network)
- 2 Draw ROI** Click + drag on canvas — voltage readout updates live on mouse release
- 3 Read Voltages** X start/end (sawtooth) + Y start/end (stair-step) shown in readout bar
- 4 Preview**  Preview Waveforms — see sawtooth + stair-step plots in Waveforms tab
- 5 Segmentation** Run Segmentation — cyan vessel overlay composited on image (~1s CPU)
- 6 Export** Copy clipboard / Save .npy / Save .csv for DAQ system

Roadmap

SHORT TERM

- Embed real GUI screenshot in April slide (once available)
- Validate ROI→voltage mapping on real galvo hardware
- Add unit tests for waveforms.py (px_to_volts round-trip, waveform shapes)
- Wire Export tab CSV to also save voltage metadata JSON alongside

MEDIUM TERM

- Replace .h5 model reference with .keras in constants.py (already the file)
- Add segmentation threshold slider (currently hardcoded 0.5)
- Multiple ROI export — waveform per ROI, indexed in .npy dict
- Restore + segment pipeline: Stage 1 ResU-Net before segmentation

LONG TERM

- Fine-tune on Andrei's chicken embryo OCTA — domain transfer
- FastAPI endpoint: POST image → JSON voltages (for integration with DAQ software)
- Live DAQ integration: send waveforms directly to NI card via nidaqmx
- Per-slab modality support: ILM_OPL vs OPL_BM ROI selection

PhotoMedicine Labs · URA

OCTA ROI Selector & Galvo Waveform Generator

PyQt5 · TensorFlow · Matplotlib · OpenCV · NumPy

```
python main.py | python main.py --image path/to/octa.png
```

OCTA-500: iee-dataport.org/open-access/octa-500

ROSE: zenodo.org/records/12775880

Model: `models/segmentation_best.h5` (or `.keras`)

Aaren Arasaratnam



Expected Results & Reference Benchmarks

Stage 2: Segmentation Targets					
Dice	> 0.80	0.80–0.85 (Giarratano 2020)	Precision	> 0.75	avoid over-segmentation
IoU	> 0.67	lower bound from Dice	Recall	> 0.80	capture fine vessels
clDice	> 0.75	topology preservation			

Stage 1: Restoration Targets (Liao et al. 2022 baseline)					
PSNR	> 23 dB	Liao et al. baseline: 24.2 dB			
SSIM	> 0.55	Liao et al. baseline: 0.59			
MAE	< 0.05	pixel-level fidelity			

If metrics fall short...

Dice < 0.60

Check mask binarization,
loss function, learning rate

val_loss > 1.0 or oscillating

Validate CLAHE was applied,
check class balance

Over-segment.

Reduce BCE_WEIGHT,
increase dropout

Under-segment.

Lower threshold < 0.5,
check CLAHE output

PSNR plateaus

Check artifact severity range,
verify perceptual loss weight

What's Next

SHORT TERM

- Clean up codebase — consolidate model-0_9.py and model.py into single versioned file
- Run check.py on combined OCTA-500+ROSE dataset to verify vessel ratios
- Validate motion artifact simulation looks realistic (check.py → visualize_motion_artifacts)
- First training run — Stage 1 restoration baseline

MEDIUM TERM

- Train Stage 2 segmentation on clean + ROSE-augmented data
- Run evaluate_1.py — generate metrics table + TP/FP/FN overlays
- Benchmark vs Giarratano 2020 (Dice 0.80–0.85) and Liao et al. 2022
- Add ROSE-2 (Heidelberg scanner) to improve cross-scanner generalisation

LONG TERM

- Fine-tune on Andrei's chicken embryo OCTA data — test domain transfer
- Explore per-slab modality inputs (ILM_OPL vs OPL_BM)
- REST API via FastAPI + Uvicorn for clinical inference endpoint
- Investigate foundation model (SAM, MedSAM) as segmentation backbone

PhotoMedicine Labs

OCTA Restoration & Segmentation

Dataset: ieee-dataport.org/open-access/octa-500

ROSE: zenodo.org/records/12775880

Ref: Liao et al. 2022 · Giarratano et al. 2020

Contributions welcome — open an issue or PR

